SNARKeling Treasure Hunt - Findings Report

Table of contents

Contest Summary

Sponsor: First Flight #59

Dates: Apr 16th, 2026 - Apr 23rd, 2026

See more contest details here

Results Summary

Number of findings:

- High: 2
- Medium: 0
- Low: 3

High Risk Findings

H-01. Treasure Secrets Stored as Plaintext Comments in Deployment Script

Root + Impact

Description

- The entire security model rests on participants **physically finding** a treasure and learning its secret. The ZK circuit exists precisely to prove knowledge of this secret without revealing it.
- The deployment script hardcodes all 10 treasure secrets in a plaintext comment block that will be committed to version control and compiled into bytecode metadata.

```
// Secret Treasures for the snorkeling hunt (not revealed to the public):
//      1, 2, 3, 4, 5, 6, 7, 8, 9, 10
@>// Treasures' hashes (revealed to the public, used as public inputs for the
proof generation):
//
150566231309314563127541858139077184792154186352784023009100711216604177550 2,
//
-787605917020763941713837706866324555936060620700057075358220870687931618335 3,
//
-560285974102256180737090051627798697051653812887195425753219763723959454105 0,
//
225668927684739934535979227740664446201472341639829021295282120594095930720 5,
//
103112101686135687921240084315807672279824464517423667712857920605566360047 70,
//    -569763786141643380748470334769940469574357004336584928079866375839506750 8,
//
-200929578987956288235928132115857381064269591347521080399148009746283210480 6,
//
893181495283985729989684031195375493178708033340530039878763751271705940690 8,
//
-441772611403917173493455978336872641319054156529152376766145238502204312455 2,
//    -961435057317293580094826482786572873533235701183329831124091847635547871092
// Note: The TreasureHunt contract requires funding at deployment, so we send 100
ETH
```

Risk

Likelihood:

- Deployment scripts are routinely pushed to public GitHub repositories.
- Even in a private repo, compiled contract bytecode embeds the IPFS/Swarm hash of the full source metadata, from which comments are recoverable.
- Secrets `1` through `10` are trivially guessable by brute force regardless of whether the file is public.

Impact:

- Any party who reads the script (or brute-forces the small secret space) can generate valid ZK proofs for all 10 treasures **without ever visiting the hunt locations**, claiming all 100 ETH before any legitimate finder can act.
-

The physical treasure hunt is rendered meaningless; the protocol's core value proposition is destroyed.

Proof of Concept

Off-chain attacker reads Deploy_s.sol and iterates over secrets 1-10

```
for (let secret of [1,2,3,4,5,6,7,8,9,10]) {
    const proof = await generateProof(secret, recipientAddress); // valid ZK proof
    await hunt.claim(proof, treasureHashes[secret-1], recipientAddress);
}
// All 100 ETH drained without physical participatioz.
```

Recommended Mitigation

Never store secrets in source code, comments, or version-controlled files. Generate secrets from a secure off-chain key management system (HSM, encrypted vault, etc.) and destroy them after their Pedersen hashes are committed to the circuit.

- Use cryptographically random 32-byte secrets (e.g., `cast keccak "$(openssl rand -hex 32)"`) rather than sequential integers.
- Add a `.gitignore` rule that excludes any file containing raw secrets from version control.
- Audit all metadata compilation outputs to ensure secret material is not embedded in bytecode CBOR/IPFS metadata.

```
- remove this code
+ add this code
```

H-02. Uninitialized Immutable _treasureHash Causes Wrong Mapping Key Same Treasure Claimable Unlimited Times

Root + Impact

Description

- The contract tracks claimed treasures via `mapping(bytes32 => bool) public claimed`. When a participant submits a valid proof, `_markClaimed(treasureHash)` writes `claimed[treasureHash] =`

`true` using the **caller-supplied** `treasureHash` parameter.

-

However, the duplicate-claim guard on line 88 reads from `claimed[_treasureHash]` — the **immutable storage variable** — which is declared but **never assigned** in the constructor, meaning it is permanently `bytes32(0)`.

- These two keys are completely independent. Unless a claimant submits `treasureHash == bytes32(0)`, the guard `claimed[_treasureHash]` always evaluates to `false`, and the same treasure can be claimed again on every subsequent call.

```
// Root cause — two different keys used for write vs. read:

bytes32 private immutable _treasureHash;   // @> Never assigned; always bytes32(0)

function claim(...) external nonReentrant() {
    ...
    // @> Checks claimed[bytes32(0)] — ALWAYS false unless treasureHash param is 0
    if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);
    ...
    _incrementClaimsCount();
    // @> Writes claimed[treasureHash_param] — correct key, but never re-read
above
    _markClaimed(treasureHash);
    ...
}
```

Risk

Likelihood:

- Any participant who obtains **one** valid proof for one treasure can re-submit it with the same `treasureHash` on every call; the guard never fires because the read key (`_treasureHash`) and the write key (`treasureHash` param) are always different.

-

No special tooling is needed — a simple loop of `claim()` calls suffices.

Impact:

- A single valid proof drains the entire contract balance in up to 10 sequential transactions (bounded only by `MAX_TREASURES`), defrauding all other treasure finders of their rewards.

-

Legitimate treasures are never recorded as claimed, so the hunt's state is completely corrupted.

Proof of Concept

Attacker has one valid proof for treasureHash = X.

claimsCount starts at 0, contract holds 100 ETH.

```
for (uint i = 0; i < 10; i++) {
    // Each iteration:
    //   claimed[bytes32(0)] == false  → guard passes
    //   claimed[X]          is set    → never read back by guard
    //   claimsCount++
    hunt.claim(proof, X, recipientAddress);  // drains 10 ETH each time
}
// After 10 calls: attacker has received 100 ETH, contract is empty.
```

Recommended Mitigation

Replace `_treasureHash` (the immutable) with `treasureHash` (the parameter) in the duplicate-claim guard, and remove the unused immutable variable:

```
- remove this code
// Remove this entirely — it serves no purpose and is never initialized:
// bytes32 private immutable _treasureHash;

+ // Before (buggy):
if (claimed[_treasureHash]) revert AlreadyClaimed(treasureHash);

// After (fixed):
if (claimed[treasureHash]) revert AlreadyClaimed(treasureHash);
```

Low Risk Findings

L-01. withdraw() Missing Access Control Anyone Can Trigger Post-Hunt Withdrawal

Root + Impact

Description

- The documented design states withdrawal of leftover funds is an **owner-controlled** admin flow.
- `withdraw()` sends the entire contract balance to `owner`, which prevents direct theft, but the function lacks the `onlyOwner` modifier, allowing **any external account** to invoke it.

```
// Root cause in the codebase with @> mark// @> No onlyOwner modifier — any caller
can trigger this
function withdraw() external {
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");
```

```
    uint256 balance = address(this).balance;
    require(balance > 0, "NO_FUNDS_TO_WITHDRAW");
    // @> Always sends to owner, so no theft — but caller is not authenticated
    (bool sent, ) = owner.call{value: balance}("");
    require(sent, "ETH_TRANSFER_FAILED");

    emit Withdrawn(balance, address(this).balance);
}s to highlight the relevant section
```

Risk

Likelihood:

- Any on-chain observer can call this function once `claimsCount == MAX_TREASURES`.
-

Bots routinely front-run state-change events to trigger permissionless functions.

Impact:

- The owner loses control over the timing of their own withdrawal — a third party forces the transfer at any moment after the hunt ends.
- Gas refund griefing: a malicious actor can repeatedly front-run the owner's own withdrawal transaction.
- Violates the principle of least privilege and the protocol's documented access control model.

Proof of Concept

After all 10 treasures are claimed:

Attacker calls withdraw() before owner does.

```
hunt.withdraw(); // Succeeds — sends balance to owner despite attacker calling
```

Recommended Mitigation

Add the `onlyOwner` modifier:

```
- remove this function withdraw() external onlyOwner {
    require(claimsCount >= MAX_TREASURES, "HUNT_NOT_OVER");
    ...
}
```

L-02. Claimed Event Emits msg.sender Instead of recipient Off-chain Tracking Broken

Root + Impact

Description

- The `Claimed` event signature is `event Claimed(bytes32 indexed treasureHash, address indexed recipient)` clearly intended to record the ETH recipient.
-

The emission inside `claim()` passes `msg.sender` (the proof submitter) as the second argument instead of the `recipient` (the payee), producing permanently incorrect on-chain logs.

```
event Claimed(bytes32 indexed treasureHash, address indexed recipient);

function claim(bytes calldata proof, bytes32 treasureHash, address payable
recipient) external ... {
    ...
    (bool sent, ) = recipient.call{value: REWARD}("");   // @> ETH goes to
recipient
    require(sent, "ETH_TRANSFER_FAILED");

    // @> Bug: emits msg.sender, not recipient
    emit Claimed(treasureHash, msg.sender);
}
```

Risk

Likelihood:

- This bug fires on every successful claim.
- The discrepancy is only visible when `msg.sender != recipient`, which is always the case (the guard `if (recipient == msg.sender) revert InvalidRecipient()` ensures they differ).

Impact:

- Block explorers, analytics dashboards, and subgraphs that index the `Claimed` event will display wrong recipient addresses.
- The event is immutable on-chain — the incorrect data cannot be corrected after deployment.
- Any off-chain system awarding secondary prizes or reputation based on event logs will attribute rewards to the wrong party.

Proof of Concept

Alice submits proof, designating Bob as recipient. ETH is correctly sent to Bob.

But the event logs Alice as recipient:

```
emit Claimed(treasureHash, alice); // Should be bob
```

Recommended Mitigation

Make changes in code as shown below

```
// Before (buggy):
emit Claimed(treasureHash, msg.sender);

// After (fixed):
emit Claimed(treasureHash, recipient);
```

L-03. updateVerifier() Does Not Validate New Verifier Address Can Be Set to address(0)

Root + Impact

Description

- The constructor correctly validates `if (_verifier == address(0)) revert InvalidVerifier()`.
- `updateVerifier()`, which replaces the verifier at runtime, performs no such check, allowing the owner to accidentally (or maliciously, post-key-compromise) set the verifier to `address(0)` or any non-contract address.

```
function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
    // @> No zero-address or code-size check on newVerifier
    verifier = newVerifier;
    emit VerifierUpdated(address(newVerifier));
}
```

Risk

Likelihood:

- Requires a compromised or negligent owner key unlikely but non-zero.

Impact:

- Setting `verifier = address(0)` causes `verifier.verify(...)` to revert on all future `claim()` calls, permanently bricking the hunt without a recovery path while the contract still holds funds.

- Even if caught quickly, the contract must be unpaused → re-paused → verifier updated, during which time the hunt is halted.

Proof of Concept

```
function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
    // @> No zero-address or code-size check on newVerifier
    verifier = newVerifier;
    emit VerifierUpdated(address(newVerifier));
}
```

Recommended Mitigation

Mirror the constructor's validation in `updateVerifier()`:

```
function updateVerifier(IVerifier newVerifier) external {
    require(paused, "THE_CONTRACT_MUST_BE_PAUSED");
    require(msg.sender == owner, "ONLY_OWNER_CAN_UPDATE_VERIFIER");
    if (address(newVerifier) == address(0)) revert InvalidVerifier();
    // Optional: verify new address has deployed code
    uint256 size;
    assembly { size := extcodesize(newVerifier) }
    require(size > 0, "VERIFIER_NOT_A_CONTRACT");
    verifier = newVerifier;
    emit VerifierUpdated(address(newVerifier));
}
```